

Counting Sort and Radix Sort: Sorting Without Comparing

Programming and Data Structures — Week 5, Lecture 3

Dr. Innocent Nyalala

Objectives

By the end of this lecture, you will be able to explain how counting sort achieves $O(n+k)$ time specifically by never directly comparing two input elements to each other; implement counting sort entirely from scratch, including the stability-preserving final placement pass; explain how radix sort extends the counting sort idea to numbers with many digits by applying it repeatedly, one digit at a time; and state precisely the conditions under which these algorithms are, and are not, a good choice compared to the comparison-based sorts covered so far this week.

Outline

Today covers an exam-grade-tally analogy motivating a fundamentally different sorting strategy; counting sort's core idea; a complete trace of counting sort, including its stability-preserving final pass; a precise analysis of why counting sort runs in $O(n+k)$ time, and the conditions under which that k term ruins its practicality; radix sort, which applies counting sort's idea repeatedly, one digit at a time; a complete trace of radix sort; the MTech-track discussion of why these algorithms sidestep the $O(n \log n)$ comparison-sort lower bound without contradicting it; an in-class quiz; and a summary.

The exam-grade-tally analogy

Picture a teacher with 200 graded exam scripts, each scored from 0 to 100. Rather than comparing scripts to each other pairwise, as every sorting algorithm covered so far in this course does, the teacher instead sets up 101 piles, one for each possible score, and drops each script directly into the pile matching its score. Reading the piles in order from 0 to 100 produces every script in fully sorted order, having made zero direct comparisons between any two scripts — the entire sorting strategy this lecture formalizes.

Counting sort: the core idea

Counting sort applies specifically when every element is an integer within a known, reasonably small range $[0, k]$ — the exam scores from 0 to 100 in the analogy. The algorithm proceeds in three steps: first, count how many times each possible value from 0 to k appears in the input; second, convert these raw counts into cumulative positions, answering “how many elements in total belong before this value”; and third, place each input element directly at its computed position in the output — an operation that involves no comparison between any two input elements whatsoever.

Counting sort, in code

```
def counting_sort(arr, k):
    count = [0] * (k + 1)
    for x in arr:
        count[x] += 1                # Step 1: raw counts

    for i in range(1, k + 1):
        count[i] += count[i - 1]    # Step 2: cumulative counts (positions)

    output = [0] * len(arr)
    for x in reversed(arr):         # Step 3: place, scanning input BACKWARD
        count[x] -= 1
        output[count[x]] = x
    return output
```

Step 1 tallies raw occurrence counts, exactly like the exam-grade piles. Step 2 transforms these into cumulative counts — `count[i]` after this step tells you how many elements are less than or equal to value `i`, which directly gives the correct final position for the last occurrence of that value. Step 3 places each element by decrementing its cumulative count and using the result as an output index; scanning the *original* input backward, rather than forward, is precisely what makes this final pass stable, as the next slide traces in detail.

Tracing counting sort completely on [2, 5, 3, 0, 2, 3], k=5

Step 1 (raw counts): `count = [1, 0, 2, 2, 0, 1]` (index = value)

Step 2 (cumulative counts): `count = [1, 1, 3, 5, 5, 6]`

Step 3 (place, scanning input BACKWARD: 3, 2, 0, 3, 5, 2):

`x=3: count[3]=5 → count[3]-=1=4 → output[4]=3 → count=[1,1,3,4,5,6]`

```

x=2: count[2]=3 → count[2]-=1=2 → output[2]=2 → count=[1,1,2,4,5,6]
x=0: count[0]=1 → count[0]-=1=0 → output[0]=0 → count=[0,1,2,4,5,6]
x=3: count[3]=4 → count[3]-=1=3 → output[3]=3 → count=[0,1,2,3,5,6]
x=5: count[5]=6 → count[5]-=1=5 → output[5]=5 → count=[0,1,2,3,5,5]
x=2: count[2]=2 → count[2]-=1=1 → output[1]=2 → count=[0,1,1,3,5,5]

```

Result: output = [0, 2, 2, 3, 3, 5]

Tracing the full three-step process on [2, 5, 3, 0, 2, 3] with $k = 5$: the raw counts show one 0, zero 1s, two 2s, two 3s, zero 4s, one 5. The cumulative counts transform this into position information. Placement then scans the original input backward — 3, 2, 0, 3, 5, 2 — decrementing each value's cumulative count and using the result as the output index. The final result, [0, 2, 2, 3, 3, 5], is correctly sorted, and this trace can be checked against the input to confirm every element appears the correct number of times.

Why scanning backward preserves stability

The two 3s in the input, at original indices 2 and 4, must retain their relative order in a stable sort. Scanning the input backward processes index 4's 3 first, placing it at output position 4 (the larger of the two final positions among 3s). It then processes index 2's 3, placing it at output position 3 (the smaller position). Since index 2 comes before index 4 in the original input, and it correctly receives the earlier output position, the relative order is preserved — this is precisely why the backward scan is not an arbitrary implementation detail, but the mechanism that makes counting sort stable.

Why $O(n+k)$, precisely — and when k ruins it

Counting sort's three steps cost $O(n)$, $O(k)$, and $O(n)$ respectively, giving a total of $O(n+k)$ — genuinely linear when k is comparable to or smaller than n . This is the entire appeal of the algorithm: it can beat the $O(n \log n)$ comparison-sort results covered in Week 6, when applicable. But this is conditional on k being small. Sorting 100 exam scores with $k = 100$ is excellent, $O(n+k)$ with both terms comparable. Sorting 100 completely arbitrary 64-bit integers, where $k = 2^{64}$, would require an absurdly large count array — this is precisely the failure mode radix sort is built to avoid.

Radix sort: applying counting sort digit by digit

Radix sort sidesteps counting sort's large- k problem by never sorting by the full value directly. Instead, it sorts by the least significant digit first, using counting sort with a small fixed range $k = 9$ (for base-10 digits) or $k = 1$ (for individual bits). Then it sorts by the next digit, and the next, continuing up to the most significant digit. This only works correctly because counting sort is stable: sorting by a more

significant digit never disturbs the correct relative order already established among numbers sharing that digit, by the earlier, less significant digit passes.

Tracing radix sort completely on [170, 45, 75, 90, 802, 24, 2]

Pass 1 (ones digit): 170,90,802,2,24,45,75 (stable counting sort on last digit)
Pass 2 (tens digit): 802,2,24,45,170,75,90 (stable counting sort on middle digit)
Pass 3 (hundreds digit): 2,24,45,75,90,170,802 (stable counting sort on first digit)
Result: [2, 24, 45, 75, 90, 170, 802] - fully sorted

Tracing radix sort on [170, 45, 75, 90, 802, 24, 2]: Pass 1 sorts stably by the ones digit, grouping 170, 90, 802, and 2 (all ending in 0 or 2) appropriately. Pass 2 sorts stably by the tens digit, refining the ordering further while never disturbing the relative order already correct from Pass 1. Pass 3 sorts stably by the hundreds digit, producing the fully correct final order, [2, 24, 45, 75, 90, 170, 802]. Each pass is a complete counting sort with $k = 9$, run exactly as many times as there are digit positions in the largest number.

Tracing Pass 1 of radix sort in full detail

Expanding Pass 1 to the actual bucket mechanics: extracting the ones digit of each number gives 0, 5, 5, 0, 2, 4, 2. A stable counting sort by this digit places 170 and 90 into bucket 0 (in that original relative order), 802 and 2 into bucket 2, 24 alone into bucket 4, and 45 and 75 into bucket 5. Reading the buckets in order from 0 to 9 produces 170, 90, 802, 2, 24, 45, 75 — matching the Pass 1 result stated earlier, now shown at the level of individual buckets.

The picture: piles by digit, three rounds

Pass 1: ones digit

170, 45, 75, 90, 802, 24, 2

buckets → 170,90 | 802,2 | 24 | 45,75 → 170, 90, 802, 2, 24, 45, 75

Pass 2: tens digit

802, 2 | 24 | 45,170 | 75 | 90 → 802, 2, 24, 45, 170, 75, 90

Pass 3: hundreds digit

2,24,45,75,90 | 170 | 802 → 2, 24, 45, 75, 90, 170, 802 (fully sorted)

Each pass is a stable counting sort on one digit; stability across passes is what makes the accumulation correct.

Watching the numbers redistribute into ten digit-buckets across three successive rounds — ones, then tens, then hundreds — makes visible exactly why stability across passes is the property that makes the whole process correct: each round refines, rather than undoes, the ordering established by the previous round.

MTech addition — the $O(n \log n)$ lower bound, and why this isn't a contradiction

A well-known and important result, proven formally in the next lecture, states that any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case — no comparison sort can ever beat this bound, no matter how cleverly designed. Counting sort's $O(n+k)$ and radix sort's $O(d(n+k))$ (for d digit passes) can appear to violate this bound when k or d is small relative to n . The resolution is that neither algorithm ever directly compares two input elements to each other at any point — they use direct indexing and digit extraction instead. The $\Omega(n \log n)$ lower bound applies specifically and only to algorithms that sort by comparison; these algorithms simply are not in that category, so there is no contradiction.

Radix sort's total cost

Radix sort's total cost sums the cost of each digit pass: d passes, each costing $O(n+k)$, giving a total of $O(d(n+k))$. Since k is fixed and small (9 for base-10 digits, or as small as 1 for individual bits), this simplifies to $O(dn)$. For fixed-width numbers — 32-bit integers processed one byte at a time need only $d = 4$ passes — d is a small constant independent of n , making the overall algorithm genuinely $O(n)$, linear time, a result no comparison-based sort can achieve in general.

Exercise

As an exercise, implement `counting_sort` and confirm it is genuinely stable by sorting a list of tagged tuples such as `(3, "a")`, `(3, "b")`, `(1, "c")` by their first element only, checking that "a" and "b" retain their original relative order in the output. Then implement `radix_sort` for base-10 non-negative integers, using your own `counting_sort` implementation as the subroutine applied at each digit position, and verify it against Python's built-in `sorted()` on several random test inputs.

Real-world use: where these actually appear

Counting sort in isolation is used directly whenever the value range is genuinely small relative to n — bucketing exam grades, ages, or small categorical codes — but its main real-world role is as the stable subroutine inside radix sort. Radix sort itself is used for sorting fixed-width keys where the digit or byte structure is known in advance: IP addresses, fixed-length string keys, and suffix arrays in text processing and bioinformatics all commonly use radix sort. Both algorithms appear frequently in specialized,

performance-critical systems code — database engines and GPU-based sorting implementations in particular — precisely because their $O(n)$ or near- $O(n)$ behavior is achievable when the input's structure is known and exploited in advance, rather than treated as arbitrary comparable data.

Quiz — is counting sort a good choice?

For each scenario, decide whether counting sort is a good choice, before checking the answer: sorting 10,000 ages, each between 0 and 120; and sorting 10,000 arbitrary 32-bit hash values.

Quiz — answer

Sorting 10,000 ages between 0 and 120 is an excellent fit for counting sort: $n = 10,000$ and $k = 120$, giving $O(n+k)$ that is essentially $O(n)$, with a tiny, practical count array. Sorting 10,000 arbitrary 32-bit hash values is a poor fit for counting sort directly: $k = 2^{32}$ would require a count array of over 4 billion slots, hopelessly impractical regardless of how few actual elements there are to sort. Radix sort, processing one byte (or a few bits) at a time across 4 passes for a 32-bit value, is the correct tool for this second scenario instead.

A note on negative numbers and non-integer keys

Both algorithms as presented assume non-negative integer keys. Negative numbers are handled by first adding the absolute value of the minimum element to every value, shifting the whole range to start at zero, sorting normally, and then subtracting the same offset back at the end. Non-integer keys are more delicate: radix sort generalizes reasonably naturally to fixed-length strings, sorting by character position much as it sorts numbers by digit position, but requires careful handling of strings of different lengths. Counting sort does not generalize directly to arbitrary non-integer keys at all — it requires first mapping keys onto a small range of integers, which is not always possible or practical.

Summary

Counting sort achieves $O(n+k)$ time by never directly comparing two input elements, instead counting occurrences and computing exact output positions directly. Scanning the original input backward during the final placement pass is specifically what preserves stability. Radix sort extends this idea to numbers with many digits by applying stable counting sort repeatedly, processing the least significant digit first and progressing to the most significant. Neither algorithm contradicts the well-known $O(n \log n)$ lower bound for comparison-based sorts, because neither one sorts by comparison at all. Next lecture proves that lower bound rigorously using a decision- tree argument, completing this week's foundation before Week 6 introduces merge sort and quicksort, the comparison-based algorithms that actually achieve $O(n \log n)$.